Resumo – Introdução à Orientação a Objetos

1. O problema: dados soltos

Até agora, nossos programas eram **procedurais**: o foco está nas ações, criamos funções e as funções operam sobre dados guardados em variáveis soltas. Isso funciona bem em programas pequenos. Mas pense em um jogo com vários personagens:

```
heroi_nome = "Heroi"
heroi_x = 100
heroi_vida = 10

goblin_nome = "Goblin"
goblin_x = 300
goblin_vida = 3

def mover(nome, x, velocidade):
    ...
```

São muitas variáveis soltas, nomes repetitivos, é fácil misturar os dados de um personagem com os de outro e as funções precisam de muitos parâmetros. A conclusão da aula: **dados que andam juntos deveriam ficar juntos.**

2. A mudança de perspectiva

A **orientação a objetos** (surgida entre o fim dos anos 60 e o início dos 70, com Simula 67 e Smalltalk) muda quem é o responsável pela ação:

	Procedural	**Orientada a objetos**
código	`mover(personagem, 10)`	`personagem.mover(10)`
ideia	a função recebe o personagem	o personagem sabe se mover
frase	"Função, faça algo com estes dados."	"Objeto, faça algo que você sabe fazer."

Macete

Você **já usa objetos** desde o começo do curso: `texto.upper()` e `lista.append(10)` são objetos executando métodos. O ponto antes do nome é o sinal: `objeto.metodo()` significa "objeto, faça isso". A novidade agora é criar os **nossos próprios** tipos de objeto.

3. Objeto: estado e comportamento

Todo objeto possui duas partes:

Estado (atributos)	Comportamento (métodos)
o que o objeto **sabe** sobre si mesmo	o que o objeto **sabe fazer**
personagem: nome, vida, posição, velocidade	personagem: mover, atacar, receber dano
lâmpada: ligada, potência	lâmpada: ligar, desligar
conta bancária: titular, saldo	conta bancária: depositar, sacar

Macete

Para separar atributos de métodos, use a gramática: **substantivos e adjetivos** (nome, saldo, ligada) viram **atributos**; **verbos** (sacar, mover, ligar) viram **métodos**. Existem várias respostas certas; o importante é separar o que o objeto sabe do que ele faz.

4. Classe: o molde

Uma **classe** é um molde para criar objetos. A classe é a **receita do bolo**; o objeto é um **bolo feito com a receita**. Com uma única classe `Ponto` podemos criar os objetos `p1`, `p2`, `p3`, cada um com seus próprios valores.

No diagrama de uma classe, os atributos ficam em cima e os métodos embaixo:

Ponto
`x: int` `y: int`
`exibir()` `mover(dx, dy)`

5. Criando a classe e os objetos

```
class Ponto:
    def __init__(self, x, y):
        self.x = x
        self.y = y


p1 = Ponto(2, 3)
print(p1.x, p1.y)
```

```
2 3
```

Linha por linha:

- `class Ponto:` cria uma nova classe. Por convenção, o nome começa com letra maiúscula;
- `__init__` é o **construtor**: executa automaticamente quando o objeto é criado. São dois sublinhados de cada lado;
- `self.x` e `self.y` são os **atributos** do objeto;
- `p1 = Ponto(2, 3)` cria um objeto usando o nome da classe. O `__init__` é chamado com `x = 2` e `y = 3`.

Atenção

Observe a diferença entre `x` e `self.x` dentro do `__init__`. O `x` é só o **parâmetro**: uma variável local que desaparece quando o construtor termina. O `self.x` é o **atributo**: fica guardado dentro do objeto. A linha `self.x = x` copia o valor recebido para dentro do objeto. Esquecer o `self.` é o erro mais comum de toda a matéria.

Manipulando atributos

Acessamos um atributo com o ponto: `objeto.atributo`. Ele pode ser lido e alterado.

```
class Ponto:
    def __init__(self, x, y):
        self.x = x
        self.y = y


p1 = Ponto(3, 4)
p2 = Ponto(10, 2)

p1.x = 5
p2.y = 3
aux = p1.y

print(f"p1 = ({p1.x}, {p1.y})   aux = {aux}")
```

```
p1 = (5, 4)   aux = 4
```

6. Métodos e o self

Um **método** é uma função definida dentro da classe. Todo método recebe `self` como primeiro parâmetro.

```
class Ponto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def exibir(self):
        print(f"Ponto: ({self.x}, {self.y})")

    def mover(self, dx, dy):
        self.x += dx
        self.y += dy


p1 = Ponto(3, 4)
p2 = Ponto(10, 2)
p1.exibir()
p2.exibir()

p1.mover(1, 2)
p1.exibir()
p2.exibir()
```

```
Ponto: (3, 4)
Ponto: (10, 2)
Ponto: (4, 6)
Ponto: (10, 2)
```

O que acontece na memória: cada objeto guarda os **seus próprios** valores de `x` e `y`. Em `p1.mover(1, 2)`, o `self` é `p1`: só `p1` muda. O `p2` continua igual.

chamada	quem é o self	o que muda
`p1.mover(1, 2)`	`p1`	`p1.x` vira 4, `p1.y` vira 6
`p2.mover(1, 2)`	`p2`	`p2.x` vira 11, `p2.y` vira 4

Macete

O `self` é **quem está antes do ponto**. Ao ler `p1.mover(1, 2)`, troque mentalmente cada `self` do método por `p1`: `self.x += dx` vira `p1.x += 1`. E repare na contagem de argumentos: o método tem 3 parâmetros (`self, dx, dy`), mas a chamada passa só 2. **O Python passa o self sozinho.**

Resumo dos conceitos

Conceito	Ideia
Classe	molde para criar objetos
Objeto	elemento criado a partir do molde
Atributo	informação guardada pelo objeto
Método	ação que o objeto sabe fazer
`self`	o próprio objeto
`__init__`	inicializa o objeto ao ser criado (construtor)

7. Resolvendo o problema do jogo

Agora os dados de cada personagem ficam juntos, dentro do objeto:

```python
class Personagem:
    def __init__(self, nome, x, y, vida, velocidade):
        self.nome = nome
        self.x = x
        self.y = y
        self.vida = vida
        self.velocidade = velocidade

    def mover_direita(self):
        self.x += self.velocidade

    def perder_vida(self):
        self.vida -= 1

    def exibir_status(self):
        print(f"{self.nome} | x = {self.x} | vida = {self.vida}")


heroi = Personagem("Heroi", 100, 200, 10, 5)
```

```
goblin = Personagem("Goblin", 300, 200, 3, 2)

heroi.mover_direita()
goblin.perder_vida()

heroi.exibir_status()
goblin.exibir_status()
```

```
Heroi | x = 105 | vida = 10
Goblin | x = 300 | vida = 2
```

Mover o herói não muda o goblin: cada objeto tem os seus próprios dados.

Atenção

A **ordem dos argumentos** na criação do objeto segue a ordem dos parâmetros do `__init__` (sem o `self`). Em `Personagem("Heroi", 100, 200, 10, 5)`, o 10 é a vida e o 5 é a velocidade. Trocar a ordem não dá erro: dá um personagem com dados errados.

Vários objetos em uma lista

Objetos podem ser guardados em listas e percorridos com `for`, como qualquer outro valor:

```
class Personagem:
    def __init__(self, nome, x, y, vida, velocidade):
        self.nome = nome
        self.x = x
        self.y = y
        self.vida = vida
        self.velocidade = velocidade

    def mover_direita(self):
        self.x += self.velocidade

    def exibir_status(self):
        print(f"{self.nome} | x = {self.x} | vida = {self.vida}")


personagens = [
    Personagem("Heroi", 100, 200, 10, 5),
    Personagem("Goblin", 300, 200, 3, 2),
    Personagem("Orc", 400, 200, 5, 1)
]

for p in personagens:
    p.mover_direita()
    p.exibir_status()
```

```
Heroi | x = 105 | vida = 10
Goblin | x = 302 | vida = 3
Orc | x = 401 | vida = 5
```

Macete

Em `for p in personagens`, a variável `p` assume, a cada volta, um objeto inteiro. Então `p.mover_direita()` chama o método daquele objeto. Os padrões de lista continuam

valendo: para achar o personagem com mais vida, compare `p.vida` e guarde o objeto `p`, e não só o número.

8. Métodos que devolvem valores e métodos que chamam métodos

Nem todo método imprime. Muitos **devolvem** uma informação com `return`, para que quem chamou decida o que fazer com ela. E um método pode chamar outro método do **mesmo** objeto usando `self`:

```
class Personagem:
    def __init__(self, nome, vida):
        self.nome = nome
        self.vida = vida

    def esta_vivo(self):
        return self.vida > 0

    def exibir(self):
        if self.esta_vivo():
            print(self.nome, "- vida:", self.vida)
        else:
            print(self.nome, "- derrotado")


p = Personagem("Slime", 2)
p.exibir()
p.vida = 0
p.exibir()
print(p.esta_vivo())
```

```
Slime - vida: 2
Slime - derrotado
False
```

Atenção

Dentro da classe, para chamar outro método use `self.esta_vivo()`, e não apenas `esta_vivo()`. Sem o `self.`, o Python procura uma função solta com esse nome e dá `NameError`.

9. Os padrões que resolvem quase tudo

Padrão 1: atributo com valor inicial fixo

Nem todo atributo vem de um parâmetro. Saldo inicial zero, lâmpada desligada, lista de notas vazia: o valor é escrito direto no `__init__`.

```
class Conta:
    def __init__(self, titular):
        self.titular = titular
        self.saldo = 0
        self.historico = []
```

```
c = Conta("Ana")
print(c.titular, c.saldo, c.historico)
```

```
Ana 0 []
```

Padrão 2: o objeto protege o próprio estado

Cada regra do enunciado vira um `if` dentro do método. O objeto não deixa que seus atributos fiquem inválidos.

```
class Item:
    def __init__(self, nome, quantidade):
        self.nome = nome
        self.quantidade = quantidade

    def exibir(self):
        print(f"{self.nome}: {self.quantidade} unidade(s)")

    def adicionar(self, qtd):
        self.quantidade += qtd

    def usar(self):
        if self.quantidade > 0:
            self.quantidade -= 1
            print(f"Usou {self.nome}")
        else:
            print(f"Nao ha {self.nome}")


pocao = Item("Pocao", 2)
pocao.exibir()
pocao.usar()
pocao.usar()
pocao.usar()
pocao.adicionar(3)
pocao.exibir()
```

```
Pocao: 2 unidade(s)
Usou Pocao
Usou Pocao
Nao ha Pocao
Pocao: 3 unidade(s)
```

Repare que o objeto **lembra** a quantidade entre uma chamada e outra: é isso que o diferencia de uma função solta.

Padrão 3: um objeto chama o método de outro

Objetos se comunicam chamando métodos uns dos outros. No ataque, **quem calcula o dano é quem recebe o ataque**.

```
class Personagem:
    def __init__(self, nome, vida, ataque, defesa):
        self.nome = nome
        self.vida = vida
```

```
        self.ataque = ataque
        self.defesa = defesa

    def esta_vivo(self):
        return self.vida > 0

    def receber_dano(self, valor):
        if not self.esta_vivo():
            print(f"{self.nome} ja esta derrotado.")
        else:
            dano_final = valor - self.defesa
            if dano_final < 1:
                dano_final = 1
            self.vida -= dano_final
            if self.vida < 0:
                self.vida = 0
            print(f"{self.nome} recebeu {dano_final} de dano")

    def atacar(self, outro):
        if not self.esta_vivo():
            print(f"{self.nome} nao pode atacar.")
        else:
            print(f"{self.nome} atacou {outro.nome}")
            outro.receber_dano(self.ataque)


heroi = Personagem("Heroi", 30, 8, 2)
goblin = Personagem("Goblin", 20, 5, 2)

heroi.atacar(goblin)
goblin.atacar(heroi)
print(heroi.vida, goblin.vida)
```

```
Heroi atacou Goblin
Goblin recebeu 6 de dano
Goblin atacou Heroi
Heroi recebeu 3 de dano
27 14
```

Macete

Em `heroi.atacar(goblin)`, dentro de `atacar` o `self` é o herói e o `outro` é o goblin. Na linha `outro.receber_dano(self.ataque)`, o herói passa o **seu** ataque (8) e o goblin usa a **sua** defesa (2) para calcular $8 - 2 = 6$. Para não se perder: escreva ao lado de cada linha quem é `self` e quem é `outro` naquele momento.

10. Erros mais comuns

Atenção

- **Esquecer o self como parâmetro:** `def exibir():` dentro da classe. Na chamada `p.exibir()`, o Python passa o objeto e dá `TypeError` ("takes 0 positional arguments but 1 was given").

- **Esquecer o self. no atributo:** `saldo += valor` em vez de `self.saldo += valor`. A variável local não existe e o atributo não muda.
- **Escrever init errado:** `_init_` (um sublinhado) ou `__int__`. O construtor não é chamado e os atributos nunca são criados.
- **Passar o self na chamada:** `p.mover(p, 1, 2)` passa argumentos demais.
- **Esquecer os parênteses:** `p.exibir` (sem parênteses) não executa o método.
- **Esquecer o return:** um método como `area()` que calcula mas não devolve faz `print(r.area())` exibir `None`.
- **Confundir classe e objeto:** `Ponto.x` não existe; o `x` é de cada objeto (`p1.x`).

11. Questões de exemplo resolvidas

Questão 1 — Atributos e métodos (exercício da aula)

Para cada objeto, indique possíveis atributos e métodos: lâmpada, celular, conta bancária e pessoa.

Uma resposta possível. Pergunte "o que ele sabe?" e "o que ele faz?".

Objeto	Atributos	Métodos
Lâmpada	ligada, potência	`ligar()`, `desligar()`
Celular	bateria, volume	`ligar()`, `silenciar()`
Conta bancária	titular, saldo	`depositar()`, `sacar()`
Pessoa	nome, idade	`falar()`, `andar()`

Questão 2 — Conta bancária com regras

Crie a classe `ContaBancaria` com titular e saldo inicial zero, um método para depositar (só valores positivos), um para sacar (só se houver saldo, devolvendo `True` ou `False`) e um para exibir.

Raciocínio. Saldo começa em zero: padrão 1. Cada regra vira um `if`: padrão 2. O `sacar` devolve um valor lógico para que quem chamou saiba se deu certo.

```
class ContaBancaria:
    def __init__(self, titular):
        self.titular = titular
        self.saldo = 0

    def depositar(self, valor):
        if valor > 0:
            self.saldo += valor
        else:
            print("Valor invalido")

    def sacar(self, valor):
        if valor <= self.saldo:
            self.saldo -= valor
            return True
        print("Saldo insuficiente")
```

```
        return False

    def exibir(self):
        print(f"{self.titular} | Saldo: R$ {self.saldo:.2f}")


conta = ContaBancaria("Ana")
conta.depositar(100)
conta.depositar(-5)
print(conta.sacar(30))
print(conta.sacar(500))
conta.exibir()
```

```
Valor invalido
True
Saldo insuficiente
False
Ana | Saldo: R$ 70.00
```

Questão 3 — Objetos em lista com padrão de maior

Dada uma lista de personagens, exiba o nome daquele com mais vida.
Raciocínio. É o padrão "maior valor" das listas, mas guardando o **objeto**.

```
class Personagem:
    def __init__(self, nome, vida):
        self.nome = nome
        self.vida = vida


grupo = [Personagem("Heroi", 10), Personagem("Orc", 15),
         Personagem("Goblin", 3)]

maior = grupo[0]

for p in grupo:
    if p.vida > maior.vida:
        maior = p

print(maior.nome, maior.vida)
```

```
Orc 15
```

Questão 4 — Leitura de código

Escreva a saída do programa abaixo.

```
class Contador:
    def __init__(self, nome):
        self.nome = nome
        self.valor = 0

    def incrementar(self, passo):
        self.valor += passo
        return self.valor
```

```
a = Contador("A")
b = Contador("B")

a.incrementar(2)
print(b.incrementar(5))
print(a.incrementar(a.valor))
c = a
c.incrementar(1)
print(a.nome, a.valor, b.valor)
```

Raciocínio. Acompanhe cada objeto separadamente.

- `a.incrementar(2)`: `a.valor` vira 2. O retorno não é impresso;
- `b.incrementar(5)`: `b.valor` vira 5, e esse valor é impresso;
- `a.incrementar(a.valor)`: o argumento é avaliado antes, valendo 2; `a.valor` vira 4;
- `c = a` **não cria um novo objeto**: `c` e `a` são o mesmo. Incrementar `c` muda `a` para 5.

```
5
4
A 5 5
```

12. Cola rápida

Quero...	**Escrevo...**
criar uma classe	`class Nome:`
construtor	`def __init__(self, a, b):`
criar atributo	`self.a = a` (dentro do `__init__`)
atributo com valor fixo	`self.saldo = 0`
criar objeto	`obj = Nome(1, 2)`
ler / alterar atributo	`obj.a` / `obj.a = 5`
criar método	`def acao(self, param):`
chamar método	`obj.acao(10)` (sem passar o `self`)
método chama método do mesmo objeto	`self.outro_metodo()`
objeto chama método de outro	`outro.receber_dano(self.ataque)`
percorrer objetos	`for p in lista: p.exibir()`